

Benchmarking Haute Performance

Multiplication de Matrices CPU vs GPU

Analyse comparative (NumPy vs CuPy)

Présenté par :

Youssef Sarraf
Sifeddine Legnioui

Encadré par :

Mme Oumaima Guendoul

Master d'Excellence en Intelligence Artificielle
Faculté des Sciences Ben M'Sik, Université Hassan II de Casablanca



Contexte

La multiplication de matrices ($C = A \times B$) est une opération fondamentale en informatique et le cœur mathématique de l'Intelligence Artificielle (Deep Learning).

Le Problème :

- C'est une opération algorithmique très lourde : complexité de $\mathcal{O}(N^3)$.
- Une simple matrice de $10\,000 \times 10\,000$ requiert **2 000 milliards** d'opérations mathématiques.
- Le temps de calcul séquentiel devient rapidement un frein à la recherche.

Comprendre la différence matérielle

CPU (Processeur Central)

- Optimisé pour le traitement **séquentiel** rapide.
- Faible nombre de cœurs (4 à 16).
- Cœurs très complexes (haute fréquence).
- Analogie : Une voiture de course.

GPU (Carte Graphique)

- Optimisé pour le traitement **parallèle massif**.
- Des **milliers** de cœurs simples.
- Exécution simultanée des instructions.
- Analogie : Un train de marchandises.

Matériel

- **Environnement** : Cloud Google Colab
- **Accélérateur** : GPU NVIDIA Tesla T4 (15 Go VRAM)

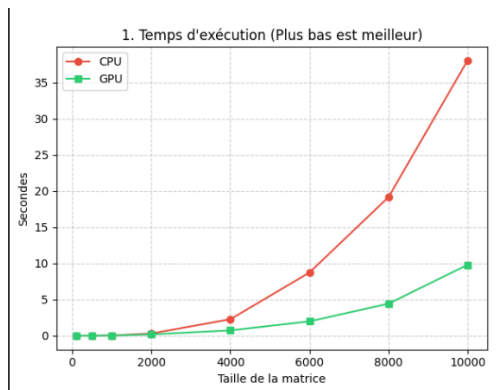
Logiciel

- **Langage** : Python 3
- **Approche CPU** : Bibliothèque NumPy (Standard de la Data Science).
- **Approche GPU** : Bibliothèque CuPy (Implémentation de l'API CUDA pour Python).

Méthodologie :

- ① Génération de matrices carrées aléatoires (de 100×100 à $10\,000 \times 10\,000$).
- ② Exécution d'un "Warmup" (Préchauffage) pour initialiser le contexte matériel CUDA et ne pas fausser les mesures.
- ③ Mesure de 3 indicateurs clés :
 - Le **temps d'exécution** (en secondes).
 - Le **facteur d'accélération** (Speedup).
 - La **puissance de calcul brute** (GigaFLOPS).

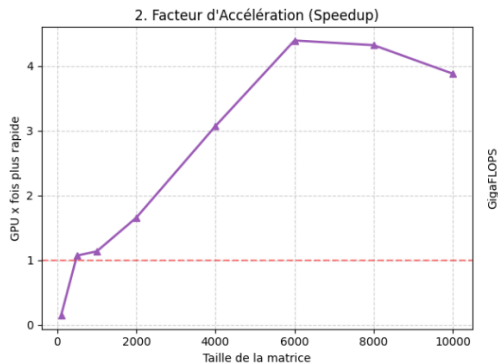
Résultat 1 : Comparaison des temps d'exécution



Observations :

- À $10\,000 \times 10\,000$, le CPU met environ 40 secondes. Le GPU met moins de 10 secondes.
- La complexité cubique étouffe le processeur central.

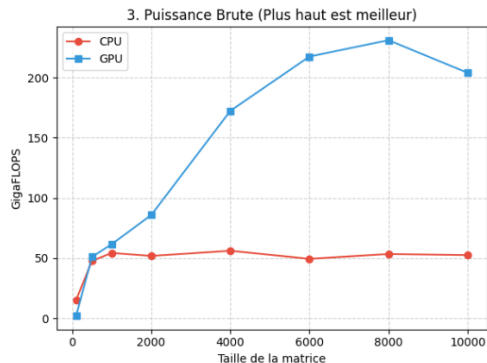
Résultat 2 : Analyse du Facteur d'Accélération (Speedup)



Observations :

- Ratio calculé : $Temps_{CPU} / Temps_{GPU}$.
- En dessous d'une taille de 1000×1000 , le CPU est parfois plus rapide !
- Le pic d'accélération montre un GPU jusqu'à **4,5 fois plus rapide**.

Résultat 3 : Puissance de calcul (GigaFLOPS)



Observations :

- Le CPU sature immédiatement autour de 50 GFLOPS.
- Le GPU a besoin de données massives pour exprimer son plein potentiel, atteignant plus de **200 GFLOPS** en pleine charge.

Le point faible du GPU

Pourquoi le GPU est-il lent sur les petites matrices?

- Le temps total du GPU n'est pas que du calcul.
- **Temps Total** = Temps de transfert (RAM $\rightarrow$ VRAM) + Temps de calcul.
- Le transfert de la mémoire via le bus de la carte mère (PCI-Express) introduit une très grande **latence**.
- Sur de petits volumes, ce voyage coûte plus cher en temps que le calcul lui-même!

Les limites matérielles (La VRAM)

La vraie contrainte de l'IA

La limite du calcul sur carte graphique aujourd'hui n'est pas la puissance de la puce, c'est la capacité de sa mémoire.

Dans notre test :

- Une seule matrice $10\,000 \times 10\,000$ (Float64) pèse environ **800 Mo**.
- L'opération nécessite 3 matrices en mémoire (A, B et le résultat C), soit **2,4 Go** de VRAM occupés.
- Notre Tesla T4 possède 15 Go, mais un modèle d'IA générative moderne nécessite facilement 80 Go.

Conclusion et Perspectives

- L'architecture parallèle (GPU) est indispensable pour traiter l'Algèbre Linéaire massive de notre époque.
- Ce n'est pas une "baguette magique" : utiliser un GPU sur de petites charges de données est contre-productif à cause du goulot d'étranglement de la mémoire.
- Le rôle de l'ingénieur est d'identifier quand la complexité du calcul justifie le coût du transfert matériel.

Merci de votre attention.